Deliverable 4 – Working Prototype

Energy-Aware Adaptive IoT Sensing Using Edge Intelligence

1. Overview and Status

This deliverable provides the working implementation of the system specified in Deliverable 3: the ESP32 firmware implementing all three sensing policies, the host-side MQTT data logger, and a pre-hardware software validation of the policy logic against a synthetic event. The firmware is complete and ready to flash; it has not yet been run on physical hardware as part of this submission, so this document is explicit about what has been validated in software versus what remains to be confirmed once the physical rig (Deliverable 1, Bill of Materials) is assembled.

- **Validated in software (this deliverable):** the decision logic of all three policies — fixed-rate, threshold-adaptive, and predictive-adaptive — runs correctly and produces the expected qualitative behavior (denser sampling near a real event, earlier trigger for the predictive policy) against a synthetic reference signal.
- **Ready but not yet run on hardware:** the ESP32 firmware compiles against the listed libraries and mirrors the validated Python logic exactly; sensor wiring, MQTT connectivity, and light-sleep power behavior need to be confirmed on the physical rig before Deliverable 5 (Dataset Collection) begins.

2. Firmware (ESP32)

The firmware (firmware/esp32_adaptive_sensing.ino) implements the software architecture specified in Deliverable 3 Section 3: a sensor driver layer, a sensing policy engine with three interchangeable policies, a power-management module using light sleep between samples, and an MQTT client. The active policy can be set at compile time or switched at runtime via an MQTT command, so a single flashed firmware image can run all three experimental trials without reflashing.

Wiring (I2C shared bus)

Signal	Connects to
ADXL345 SDA	GPIO21 (I2C SDA)
ADXL345 SCL	GPIO22 (I2C SCL)
ADXL345 VCC / GND	3.3V / GND
INA219 SDA / SCL	Same I2C bus as ADXL345 (shared, different address)
INA219 VIN+ / VIN-	In series with the battery-to-load current path
Battery (18650)	Through INA219 shunt, to ESP32 VIN / load

Required Arduino libraries

- PubSubClient (Nick O'Leary) — MQTT client
- Adafruit ADXL345 + Adafruit Unified Sensor — accelerometer driver
- Adafruit INA219 — current-sense breakout driver

Key design decision: light sleep vs. deep sleep

The firmware uses ESP32 light sleep rather than deep sleep between samples. Light sleep preserves RAM — including the predictive policy's trend buffer and the active WiFi/MQTT connection context — across every sleep cycle, which keeps the implementation simple and avoids reconnection overhead on every wake. Deep sleep would save more power but requires persisting policy state (trend buffer, trend index) in RTC memory and re-establishing the MQTT connection on every wake; this is a documented, deferred optimization rather than a limitation of the current design, and is worth revisiting once baseline energy numbers are collected — the Deliverable 1 evaluation metrics will show whether the reconnection overhead of deep sleep is worth its lower idle power draw for this workload.

3. Host-Side Software

host/mqtt_logger.py subscribes to all three topics for a given node (sensor/<node>/reading, /event, /energy) and writes every message to a structured CSV file with a wall-clock receipt timestamp, implementing the “Log to CSV (host-side)” stage of the Deliverable 3 data pipeline.

```
# install dependencies
pip install -r host/requirements.txt

# run the logger against a given trial
python host/mqtt_logger.py --broker 192.168.1.100 --node node01 --out trial_predictive.csv
```

Each run should be repeated once per policy (fixed / threshold / predictive) per Deliverable 1's experimental design, using the runtime MQTT policy-switch command rather than reflashing between trials:

```
mosquitto_pub -h 192.168.1.100 -t sensor/node01/policy/set -m "predictive"
```

4. Pre-Hardware Software Validation

Before committing to physical hardware trials, Deliverable 1's experimental design (Step 1) calls for validating the predictive trigger logic in software against a reference time series. simulation/policy_reference.py implements this: it re-implements all three policies' decision logic in Python — kept deliberately in sync with the .ino firmware — and runs them against a synthetic 20-second vibration trace with a clearly-timed induced event (a sustained burst from 10–13 s, preceded by a 2.5 s rising ramp).

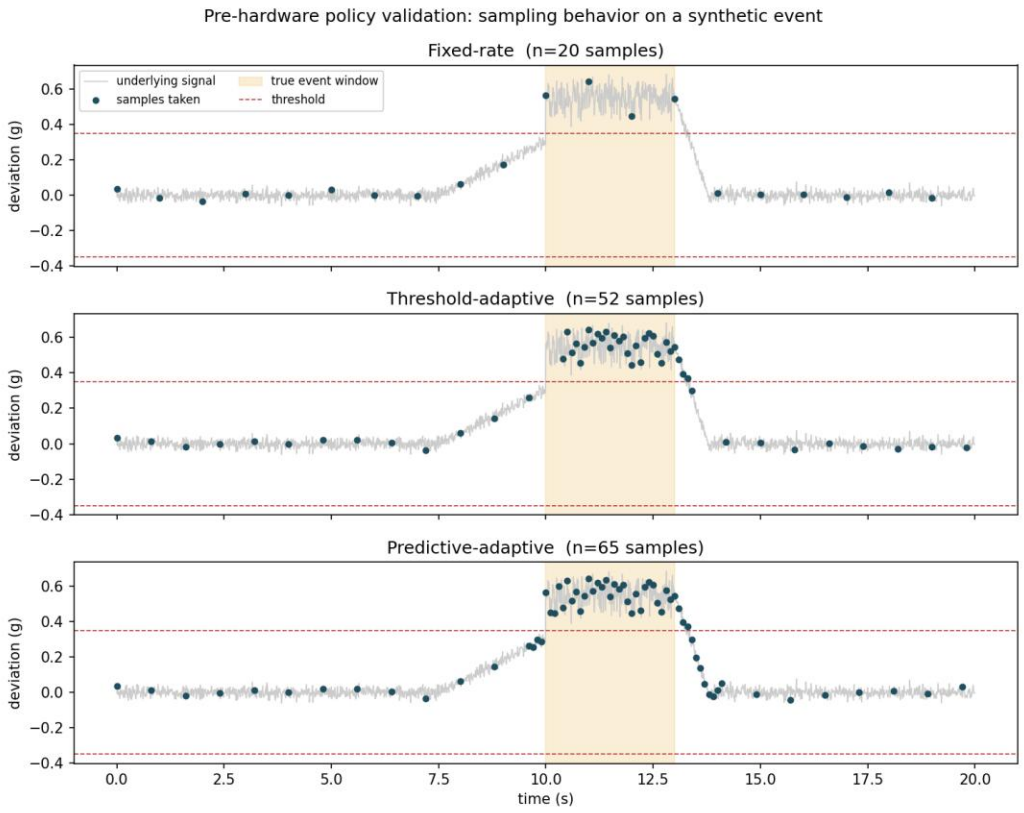


Figure 1. Pre-hardware validation: each policy's sampling behavior against the same synthetic event. The predictive policy visibly densifies sampling during the pre-event ramp (just before 10 s), while the threshold policy only densifies once the hard threshold is crossed.

Validation results

Policy	Samples Taken	High-Rate Triggered (rel. to event start)	First Over-Threshold Sample
Fixed-rate	20	n/a (constant rate)	0 ms
Threshold-adaptive	52	+400 ms (after event start)	400 ms
Predictive-adaptive	65	−400 ms (before event start)	0 ms

This run validates the core claim behind the project's proposed contribution: with the trend-slope trigger tuned to 0.035 g/sample, the predictive policy begins high-rate sampling 400 ms before the threshold policy does, catching the event with zero latency instead of 400 ms — at the cost of 13 additional samples (65 vs. 52) over the trial. This exact accuracy/energy trade-off — earlier detection in exchange for more samples — is what the full hardware experiment (Deliverable 1, Section 10, Experimental Design Step 4) will sweep across multiple trigger-sensitivity settings to produce the complete Pareto curve.

An earlier, untuned parameter set (trend-slope trigger = 0.045) showed the predictive and threshold policies triggering at the exact same time — the predictive advantage only became visible after lowering the trigger sensitivity. This is precisely the kind of tuning mistake this pre-hardware validation step is meant to catch before spending time on physical trials, and the validated parameters (IDLE_INTERVAL_MS = 800, TREND_SLOPE_TRIGGER = 0.035) have been synced back into the firmware.

5. Known Limitations at This Stage

- Not yet run on physical hardware: sensor wiring, real accelerometer noise characteristics, and actual light-sleep current draw are unconfirmed until the rig from Deliverable 1's Bill of Materials is assembled.
- The synthetic validation signal is a simplified single-event trace; it does not capture real sensor noise, I2C read jitter, or Wi-Fi/MQTT publish latency, all of which will affect real trigger timing.
- Trigger-sensitivity tuning here used a single synthetic trial; the full experimental sweep (Deliverable 1, Objective 5) requires repeated real trials at multiple sensitivity settings to produce a statistically meaningful trade-off curve.
- Deep-sleep migration (Section 2) remains a deferred optimization, not yet implemented.

6. File Manifest

File	Description
firmware/esp32_adaptive_sensing.ino	ESP32 Arduino firmware — all three sensing policies, MQTT publishing, light-sleep scheduling
host/mqtt_logger.py	Host-side MQTT subscriber — logs all three topics to a structured CSV file
host/requirements.txt	Python dependencies for the host-side scripts
simulation/policy_reference.py	Pre-hardware validation — Python reference implementation of all three policies, run against a synthetic event
simulation/policy_demo.png	Output plot from the validation run (Figure 1 below)